

Documentation interfaces interfaz_remix

This document contains the description of the interfaces used in the file `main_interfaz.f90` which is the main file of the code `interfaz_remix`.

The repository can be found at https://github.com/jordipg10/interfaz_remix.

Note: the variables that contain an asterisk are **optional** arguments.

read_clean_line

- **Description:** Internal subroutine that reads the next non-blank, non-comment line from standard input. It skips empty lines and full-line comments (lines whose first non-blank character is "!" or "#"), so the input file driven by stdin redirection can be annotated. The caller then parses the returned buffer with an internal list-directed read (e.g. `read(buf, *, iostat=ios) value`), which preserves the original type-safe conversion to integers, reals and character strings.
- **Class:** Internal procedure of program `main_interfaz` (contained subroutine).
- **Input: none.**
- **Output:**

Variable	Type	Description
<code>line</code>	String	Next clean input line, left-adjusted and with leading blanks stripped
<code>ios</code>	Integer	I/O status (0 on success, non-zero on read error or end-of-file)

- **Process:**
 - Reads one line from standard input into `line` using a character-format read `read(*, '(A)', iostat=ios) line`. On read error or end-of-file, `ios` is propagated to the caller and the subroutine returns immediately.
 - Left-adjusts the line with `adjustl` and computes its trimmed length with `len_trim`. Blank lines are skipped (the loop reads the next line).
 - If the first non-blank character is "!" or "#", the line is treated as a full-line comment and skipped. Only full-line comments are stripped: "!" or "#" appearing in the middle of a line are kept verbatim, so file paths containing those characters are preserved. Comments must therefore be placed on their own line.
 - Returns the trimmed, left-adjusted line to the caller in `line` with `ios = 0`.

read_chemistry_interface

- **Description:** Subroutine that reads all the chemical input data (species, reactions, initial and external water types, mineral zones, etc.) by dispatching to the appropriate format-specific reader.
- **Class: Chemistry**
- **Input:**

Variable	Type	Description
root	String	Root of input and output files of problem to be solved
path_pb	String	Path to input and output files of problem to be solved
path_DB	String	Path to chemical databases
num_tar	Integer	Expected number of target (domain) waters from the mesh

- **Pre-process:**
 - Sets arbitrary file units for the chemical system, computation options and local chemistry input files.
- **Process:**
 - The input format option is currently hard-coded to the CHEPROO-based format (`read_opt = 1`), which reads the chemical data through `read_chemistry_CHEPROO`; the PHREEQC (`read_opt = 2`) and PFLOTRAN (`read_opt = 3`) branches abort execution with an error stop.

write_u_init

- **Description:** Subroutine that writes a human-readable summary file describing the aqueous component definitions in terms of variable activity species, the list of aqueous variable activity species, and the aqueous component and variable activity species concentrations of every target water and of every external (boundary/recharge) water. The format reproduces the layout expected by the REMIX interface. The reference basis is the first target water (`tar_wat_indices(1)`): all component definitions, the species list and every concentration column (domain and external) are expressed in this single basis, and waters are written in order (no per-reactive-zone grouping and no name-based filtering).
- **Class: Chemistry**
- **Input:**

Variable	Type	Description
<code>path</code>	String	Path to the output file
<code>filename</code>	String	Output file name

- **Process:**
 - Takes the first target water (`tar_wat_indices(1)`) as reference: its aqueous component matrix `comp_mat_aq` defines the number of components, the variable activity species order and the component definitions. The reference species names are stored in component-matrix column order (primary species first, then secondary variable activity species).
 - For every water written (target and external), reorders its variable-activity-species concentrations to the reference species order by exact name matching (species absent from a water contribute zero) and computes its aqueous components as the product of the reference component matrix and the reordered species vector.
 - Opens the output file (`path//filename`, unit 20, status `replace`) and writes a header with the number of aqueous components.
 - Writes the list of aqueous variable activity species under the heading "Aqueous variable activity species (rows 1 to N: primary; rows N+1 to M: secondary):", where N is the number of primary species and M the total number of variable activity species (or "Aqueous variable activity species (all primary):" when there are no secondary variable activity species), one numbered entry per species.
 - Writes the aqueous components as linear combinations of the variable-activity species defined by `comp_mat_aq`, for example $u_6 = \text{hco}_3^- + \text{co}_3^{2-}$. Unit coefficients are omitted, signs are written as + / -, and non-unit coefficients are written with two decimals as `coeff*species`.

- For the target waters, writes two scientific-notation tables under the headings (verbatim) "Aqueous variable activity species concentrations in the domain (rows = species, cols = target waters):" followed by "Aqueous component concentrations in the domain (rows = components, cols = target waters):". Columns follow the order of `tar_wat_indices`. Numeric format is ES15.5.
- For the external waters, writes the corresponding species and component tables under the headings "Aqueous variable activity species concentrations of external waters (rows = species, cols = external waters):" and "Aqueous component concentrations of external waters (rows = components, cols = external waters):", each preceded by a header row of right-aligned water names in 15-character fields. Columns follow the order of `ext_waters_indices`. Numeric format is ES15.5.
- Closes `filename`

get_num_aq_comps_tar_wat

- **Description:** Function that returns the number of aqueous components in a target water. If no index is provided, the first target water is used.
- **Class: Chemistry**
- **Input:**

Variable	Type	Description
ind_tw*	Integer	Optional index of a target water

- **Output:**

Variable	Type	Description
num_aq_comps	Integer	Number of aqueous components in the target water

- **Process:**
 - Validates the input: if `ind_tw` is present and out of range, execution is aborted with "get_num_aq_comps_tar_wat: ind_tw out of range"; if it is absent and no target waters are available, execution is aborted with "get_num_aq_comps_tar_wat: no target waters available".
 - Returns the value of `num_aq_prim_species` in the `speciation_alg` object of the reactive zone associated to the target water. This local count may differ from the chemical-system-wide count when the reactive zone defines a reduced set of equilibrium reactions.

interfaz_comps_arch_eq

- **Description:** Subroutine that solves a reactive mixing iteration with components, and writes aqueous component and variable activity species concentrations, using files as arguments. It assumes that there are equilibrium reactions only (no kinetics), so no time integration is required (the time step argument is unused) and the aqueous component concentrations after reactive mixing are identical to those after conservative transport; the subroutine only speciates them.
- **Class: Chemistry**
- **Input:**

Variable	Type	Description
path	String	Path to input and output files.
num_aq_comps	Integer	Nº aqueous components.
file_in	String	File containing concentrations of aqueous components after solving a conservative transport iteration ($\tilde{\mathbf{U}}^{k+1}$).
Delta_t	Real	Time step (Δt^k)
file_out	String	File where aqueous component concentrations ($\mathbf{U}^{k+1}$) and variable activity species concentrations ($\mathbf{C}_v^{k+1}$) after solving a reactive mixing iteration will be written.

- **Pre-process:**
 - Allocate aqueous component concentrations after conservative transport ($\tilde{\mathbf{U}}^{k+1}$)
 - Determine, from the first target water, the number of variable activity species `n_nc`.
 - Allocate the variable activity species concentrations `conc_nc` with shape (`n_nc`, `num_target_waters`).
- **Process:**
 - **Read** $\tilde{\mathbf{U}}^{k+1}$ from `file_in`. **Note:** The rows must represent components and the columns targets.
 - **Solve** reactive mixing iteration for each target water j :
 - **Loop over number of target waters**
 - Speciate to compute variable activity species concentrations $\mathbf{C}_{v,j}^{k+1}$ from `u_tilde`, using Newton-Raphson.
 - Check Newton convergence: if speciation does not converge for a target water, the target index and a "No convergence in speciation after reactive mixing iteration. Try reducing the time step" message are printed and execution is aborted with an error stop.
- **Post-process:**
 - **Write** $\mathbf{U}^{k+1}$ in `file_out` (same dimensions as $\tilde{\mathbf{U}}^{k+1}$)

- **Write** c_v^{k+1} in **file_out** (rows: species, columns: targets)
- The **file_out** contents are written with these headings (verbatim): "Aqueous component concentrations after solving reactive mixing iteration (rows: components, columns: targets):" and "Concentrations of variable activity species after solving reactive mixing iteration (rows: species, columns: targets):". Numeric format is ES15.5.
- **Deallocate** arrays

interfaz_comps_arch_eq_T

- **Description:** Transposed-input variant of `interfaz_comps_arch_eq`. Solves a reactive mixing iteration with components and writes aqueous component and variable activity species concentrations, using files as arguments, when there are equilibrium reactions only (no kinetics). The only difference with respect to `interfaz_comps_arch_eq` is that the input file `file_in` has rows as targets and columns as components (transposed with respect to the canonical layout). It is selected at run time by the driver when `flag_transpose = 1`. Inputs, outputs, pre-process, process and post-process are otherwise identical to `interfaz_comps_arch_eq`, except that `file_in` is read row by row and transposed in memory before the speciation step.
- **Class: Chemistry**

interfaz_comps_arch_eq_kin

- **Description:** Subroutine that solves a reactive mixing iteration with components, and writes aqueous component concentrations, variable activity species concentrations and kinetic reaction rates, using files as arguments. It assumes that there are both equilibrium and kinetic reactions. The reactive mixing is solved implicitly: the coupled equilibrium + kinetic system is solved with Newton's method, a Crank-Nicolson temporal weighting factor $\theta = 0.5$ and a lumped reaction mixing ratio $\lambda_r = 1$. If the implicit Newton solver does not converge for a target water (stiff cell), an explicit-Euler update is used as fallback for that cell, so the run always completes.
- **Class: Chemistry**
- **Input:**

Variable	Type	Description
path	String	Path to input and output files.
num_aq_comps	Integer	Nº aqueous components.
file_in	String	File containing concentrations of aqueous components after solving a conservative transport iteration ($\tilde{\mathbf{U}}^{k+1}$).
Delta_t	Real	Time step (Δt^k)
file_out	String	File where aqueous component concentrations ($\mathbf{U}^{k+1}$) and variable activity species concentrations ($\mathbf{C}_v^{k+1}$) and kinetic reaction rates after solving a reactive mixing iteration will be written.

- **Pre-process:**
 - Allocate aqueous component concentrations after conservative transport ($\tilde{\mathbf{U}}^{k+1}$).
 - Allocate chemical reactions contribution to aqueous component concentrations ($\mathbf{U}_K^{k+1}$) (used only in the explicit-Euler fallback).
 - Determine, from the first target water, the number of kinetic reactions (aqueous kinetic `n_aq_kin`, mineral kinetic `n_min_kin`, gas kinetic `n_gas_kin`), the number of primary species `n_p`, and the number of variable activity species `n_nc`.
 - Allocate the variable activity species concentrations `conc_nc` with shape (`n_nc`, `num_target_waters`).
 - Allocate the kinetic reaction rates `rk_out` with shape (`n_aq_kin + n_min_kin + n_gas_kin`, `num_target_waters`) and initialise them to zero.
 - Build the kinetic reaction name labels `react_names` in the same row order as `rk_out`: aqueous linear kinetic, aqueous redox kinetic, kinetic mineral names (taken from the chemical system through the mineral zone), and gas kinetic

reactions (labelled `gas_kin_<i>`). They are taken from the first target water, since the chemical system is shared across all target waters.

- **Process:**

- **Read** $\tilde{\mathbf{U}}^{k+1}$ from `file_in`. **Note:** The rows must represent components and the columns targets.
- **Solve** reactive mixing iteration for each target water j :
 - **Loop over number of target waters**
 - Speciate the mixed components ($\tilde{\mathbf{U}}^{k+1}$) to obtain the mixed variable activity species concentrations $\hat{c}_{v,j}$ (initial guess: current primary species concentrations), using Newton-Raphson. If this speciation does not converge, the target water index and a "No convergence speciating `u_tilde` before the reactive mixing iteration. Try reducing the time step." message are printed and execution is aborted with an error stop.
 - Implicit solve: call `Newton_EI_eq_kin_anal_ideal_opt2` with the mixed components ($\tilde{\mathbf{U}}^{k+1}$), lumped reaction mixing ratio $\lambda_r = 1$, time step (Δt^k) and temporal weighting factor $\theta = 0.5$ (Crank-Nicolson), using $\hat{c}_{v,j}$ as initial guess, to obtain $\mathbf{c}_{v,j}^{k+1}$.
 - If the implicit Newton converges, recover the aqueous component concentrations as `u_new := comp_mat_aq * conc_nc` (restricted to the aqueous variable activity species) and skip the explicit-Euler fallback.
 - Explicit-Euler fallback (only executed when the implicit Newton does not converge, e.g. a stiff cell where dissolved O2 is driven to ~ 0): compute chemical reactions contribution to aqueous component concentrations using Euler explicit and lumping:

$$\mathbf{u}_{K,j}^{k+1} := \Delta t^k \mathbf{W} \mathbf{S}_{K,v}^T \mathbf{r}_{K,j}^k$$
 - Update the total aqueous component concentrations in target water: $\mathbf{u}_j^{k+1} = \tilde{\mathbf{u}}_j^{k+1} + \mathbf{u}_{K,j}^{k+1}$.
 - Clamp species-equivalent components: a component whose row in `comp_mat_aq` has a single +1 entry is itself a species concentration, so clamp its value to 0 if negative (explicit-Euler overshoot guard). Components with several nonzero entries are left untouched, since they are linear combinations and may legitimately be negative.
 - Speciate to compute variable activity species concentrations $\mathbf{c}_{v,j}^{k+1}$ from the updated components, using Newton-Raphson with the primary block of $\hat{c}_{v,j}$ as initial guess.
 - Check Newton convergence: if the fallback speciation does not converge for a target water, the target index and a "No

convergence in EE-fallback speciation after reactive mixing" message are printed and execution is aborted with an error stop.

- Recompute u_{new} from the speciated state: $u_{\text{new}} := \text{comp_mat_aq} * c_v$ (restricted to the aqueous variable-activity species). This keeps u consistent with c_v after the clamping step - a clamped species-equivalent component would otherwise disagree with the small positive species value recovered by Newton (e.g. $\sim 1e-15$ for dissolved O_2).
- Capture the kinetic reaction rates `rk_out` for the current target water evaluated at the end-of-step (converged) concentrations via `compute_rk_new`. Row order: aqueous (linear+redox) | minerals | gas; gas kinetic rates are not yet implemented, so their rows remain zero.

- **Post-process:**

- **Write** U^{k+1} in `file_out` (same dimensions as $\tilde{U}^{k+1}$)
- **Write** C_v^{k+1} in `file_out` (rows: species, columns: targets)
- **Write** kinetic reaction rates `rk_out` in `file_out` (rows: reactions [aqueous (linear+redox) | minerals | gas], first column: reaction name, remaining columns: targets)
- The `file_out` contents are written with these headings (verbatim): "Aqueous component concentrations after solving reactive mixing iteration (rows: components, columns: targets):", "Concentrations of variable activity species after solving reactive mixing iteration (rows: species, columns: targets):", and "Kinetic reaction rates (rows: reactions [aqueous (linear+redox) | minerals | gas], first column: reaction name, remaining columns: targets):". Numeric format is ES15.5; reaction names are left-padded into a 30-character first column.
- **Deallocate** arrays

interfaz_comps_arch_eq_kin_T

- **Description:** Transposed-input variant of `interfaz_comps_arch_eq_kin`: the input file `file_in` has rows as targets and columns as components (transposed with respect to the canonical layout) and is read row by row and transposed in memory before solving. It is selected at run time by the driver when `flag_transpose = 1`. The algorithm is otherwise identical to `interfaz_comps_arch_eq_kin`: the reactive mixing is solved implicitly (Newton with Crank-Nicolson temporal weighting factor $\theta = 0.5$ and lumped reaction mixing ratio $\lambda_r = 1$), with the same initial speciation of the mixed components (and its abort message), the same explicit-Euler fallback for cells where the implicit Newton does not converge (including the species-equivalent clamping, the re-speciation with its abort message and the component recomputation), and the same end-of-step kinetic reaction rates via `compute_rk_new`. Inputs, outputs and the output file layout (headings, ES15.5 format, 30-character reaction-name column) are also identical.
- **Class: Chemistry**

interfaz_esp_arch

- **Description:** Subroutine that solves a reactive mixing iteration with **aqueous species**, and writes aqueous species concentrations and kinetic reaction rates, using **files** as arguments. The integration method is **Euler explicit** and the reaction mixing ratios are **lumped**. It assumes that there are no equilibrium reactions.
- **Class: Chemistry**
- **Input:**

Variable	Type	Description
path	String	Path to input and output files.
num_aq_comps	Integer	Nº aqueous species (= nº aqueous components)
file_in	String	File containing concentrations of aqueous species after solving a conservative transport iteration ($\tilde{\mathbf{c}}^{k+1}$)
Delta_t	Real	Time step (Δt^k)
file_out	String	File where aqueous species concentrations ($\mathbf{c}^{k+1}$) and kinetic reaction rates after solving a reactive mixing iteration will be written

- **Pre-process:**
 - Allocate aqueous species concentrations after conservative transport ($\tilde{\mathbf{c}}^{k+1}$)
 - Allocate chemical reactions contribution to aqueous species concentrations ($\mathbf{c}_K^{k+1}$)
 - Determine, from the first target water, the number of kinetic reactions (aqueous kinetic **n_aq_kin**, mineral kinetic **n_min_kin**, gas kinetic **n_gas_kin**).
 - Allocate the kinetic reaction rates **rk_out** with shape ($\max(\text{n_aq_kin} + \text{n_min_kin} + \text{n_gas_kin}, 1), \text{num_target_waters}$) and initialise them to zero.
- **Process:**
 - **Read** $\tilde{\mathbf{c}}^{k+1}$ from **file_in**. **Note:** The rows must represent species and the columns targets.
 - **Solve** reactive mixing iteration for each target water j :
 - **Loop over number of target waters**
 - Compute chemical reactions contribution to aqueous species concentrations using **Euler explicit** and **lumping**:
$$\mathbf{c}_{K,j}^{k+1} := \Delta t^k \mathbf{S}_{K,aq}^T \mathbf{r}_{K,j}^k$$
 - Update the total aqueous species concentrations in target water:
$$\mathbf{c}_j^{k+1} = \tilde{\mathbf{c}}_j^{k+1} + \mathbf{c}_{K,j}^{k+1}$$
 - Capture the kinetic reaction rates **rk_out** for the current target water from the aqueous and mineral kinetic rate arrays of the chemistry object (Euler-explicit uses **rk_old**; gas kinetic rates not yet implemented).

- **Post-process:**

- **Write** $\mathbf{c}^{k+1}$ in `file_out` (rows: species, columns: targets)
- Write kinetic reaction rates `rk_out` in `file_out` (rows: reactions [aqueous (linear+redox) | minerals], columns: targets); unlike the component-based interfaces, no reaction-name column is written.
- The `file_out` contents are written with these headings (verbatim): "Aqueous species concentrations after solving reactive mixing iteration (rows: species, columns: targets):" and "Kinetic reaction rates (rows: reactions [aqueous (linear+redox) | minerals], columns: targets):". Numeric format is ES15.5. If there are no kinetic reactions, the line "(no kinetic reactions)" is written instead of the rates table.
- **Deallocate** arrays